

SUPABASE POSTGRES SQL QUERY DEMONSTRATION MANUAL

Live Query Execution, Relational JOIN Operations, DDL Schemas & Practical SQL Testing for ResolveDesk

Target Platform:	ResolveDesk Customer Support Platform
Database System:	Supabase Cloud (PostgreSQL 15+ Instance)
Interface Used:	Supabase Studio SQL Editor & psql CLI Client
Document Type:	Practical Query Execution & Verification Manual
Prepared By:	Viraj (Lead Author)
Institution / Dept:	J.J College of Engineering and Technology — Dept of IT
Date of Execution:	September 2026

MANUAL PURPOSE:

This document provides step-by-step SQL scripts to execute directly in the **Supabase Studio SQL Editor**. It includes Table DDL Creation, Referential Data Insertion, Multi-table JOIN Queries (INNER, LEFT, RIGHT, FULL, NATURAL), Composite Key Validation, and Intentional Error Debugging.

1. Guide: Running Queries in Supabase Studio

To execute the SQL queries demonstrated in this manual within your managed Supabase project environment, follow these steps:

Step 1: Log in to the **Supabase Dashboard** (<https://supabase.com/dashboard>) and select your project (Helpdesk Capstone).

Step 2: Click on the **SQL Editor** icon in the left navigation sidebar menu.

Step 3: Click on **+ New query** to open a clean SQL script panel.

Step 4: Copy any SQL query block from this document into the editor panel.

Step 5: Click the green **Run** button (or press Ctrl + Enter) to execute the query.

Step 6: View the formatted query result grid or execution logs in the bottom output tab.

2. Table DDL Creation Queries (Supabase PostgreSQL)

Run the following DDL script in Supabase SQL Editor to construct the primary relational tables with BIGINT identity sequences and foreign key constraints:

```
-- 1. Create USERS Table
CREATE TABLE IF NOT EXISTS public.users (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(150) UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  role VARCHAR(30) NOT NULL CHECK (role IN ('customer', 'agent', 'admin')),
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 2. Create TICKETS Table
CREATE TABLE IF NOT EXISTS public.tickets (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  customer_id BIGINT NOT NULL REFERENCES public.users(id) ON DELETE CASCADE,
  title VARCHAR(200) NOT NULL,
  category VARCHAR(50) NOT NULL,
  priority VARCHAR(20) DEFAULT 'medium',
  description TEXT NOT NULL,
  status VARCHAR(30) DEFAULT 'open' CHECK (status IN ('open', 'assigned', 'in_progress', 'resolved', 'closed')),
  assigned_agent_id BIGINT NULL REFERENCES public.users(id) ON DELETE SET NULL,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- 3. Create RESPONSES Table
CREATE TABLE IF NOT EXISTS public.responses (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  ticket_id BIGINT NOT NULL REFERENCES public.tickets(id) ON DELETE CASCADE,
  sender_id BIGINT NOT NULL REFERENCES public.users(id) ON DELETE CASCADE,
  message TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

3. Sample Data Insertion & Verification Queries

Execute data insertion in strict dependency order (parent entities first):

```
-- Insert Test Users
INSERT INTO public.users (id, name, email, password_hash, role) VALUES
(1, 'System Admin', 'admin@resolvedesk.com', '$2a$10$hash1', 'admin'),
(2, 'Agent RBNV', 'agent.rbnv@resolvedesk.com', '$2a$10$hash2', 'agent'),
(3, 'Agent X', 'agent.x@resolvedesk.com', '$2a$10$hash3', 'agent'),
(4, 'Viraj Customer', 'viraj@gmail.com', '$2a$10$hash4', 'customer'),
(5, 'Customer A', 'customer.a@gmail.com', '$2a$10$hash5', 'customer'),
(6, 'Customer B', 'customer.b@gmail.com', '$2a$10$hash6', 'customer')
ON CONFLICT (id) DO NOTHING;

-- Insert Test Tickets
INSERT INTO public.tickets (id, customer_id, title, category, priority, description, status, assigned_agent_id) VALUES
(101, 4, 'Payment failure on checkout', 'Billing', 'high', 'Card declined during payment', 'in_progress', 3),
(102, 5, 'Broken Authentication', 'Security', 'high', 'OTP email not received', 'assigned', 2),
(103, 5, 'Test Transition', 'General', 'low', 'Testing status movement', 'resolved', 2),
(104, 4, 'Billing issue', 'Billing', 'medium', 'Invoice duplicate charge', 'open', NULL)
ON CONFLICT (id) DO NOTHING;
```

Record Count Verification Query:

```
SELECT 'users' AS table_name, COUNT(*) AS record_count FROM public.users
UNION ALL
SELECT 'tickets' AS table_name, COUNT(*) AS record_count FROM public.tickets
UNION ALL
SELECT 'responses' AS table_name, COUNT(*) AS record_count FROM public.responses;
```

table_name	record_count
users	6
tickets	4
responses	2

4. Relational JOIN Query Demonstrations

4.1 INNER JOIN (Matching Customers & Submitted Tickets)

```
SELECT u.name AS customer_name, u.email, t.id AS ticket_id, t.title, t.status
FROM public.users u
INNER JOIN public.tickets t ON u.id = t.customer_id;
```

customer_name	email	ticket_id	title	status
Viraj Customer	viraj@gmail.com	101	Payment failure on checkout	in_progress
Customer A	customer.a@gmail.com	102	Broken Authentication	assigned
Customer A	customer.a@gmail.com	103	Test Transition	resolved
Viraj Customer	viraj@gmail.com	104	Billing issue	open

4.2 LEFT JOIN (All Users & Associated Tickets if Available)

```
SELECT u.name AS user_name, u.role, t.title AS ticket_title
FROM public.users u
LEFT JOIN public.tickets t ON u.id = t.customer_id;
```

user_name	role	ticket_title
System Admin	admin	NULL
Agent RBNV	agent	NULL
Agent X	agent	NULL
Viraj Customer	customer	Payment failure on checkout
Customer A	customer	Broken Authentication
Customer B	customer	NULL

4.3 RIGHT JOIN (Tickets & Assigned Agent Details)

```
SELECT t.id AS ticket_id, t.title, u.name AS assigned_agent
FROM public.users u
RIGHT JOIN public.tickets t ON u.id = t.assigned_agent_id;
```

ticket_id	title	assigned_agent
101	Payment failure on checkout	Agent X
102	Broken Authentication	Agent RBNV
103	Test Transition	Agent RBNV
104	Billing issue	NULL (Unassigned)

5. Intentional SQL Error & Integrity Constraint Testing

Testing database constraint enforcement in Supabase PostgreSQL by executing invalid operations:

Test Case 1: Foreign Key Violation (Inserting Ticket for Non-existent User)

```
INSERT INTO public.tickets (customer_id, title, category, description)
VALUES (9999, 'Orphan Ticket Test', 'General', 'Testing FK error');

ERROR: insert or update on table "tickets" violates foreign key constraint "tickets_customer_id_fkey"
DETAIL: Key (customer_id)=(9999) is not present in table "users".
```

Test Case 2: UNIQUE Constraint Violation (Duplicate Email Insertion)

```
INSERT INTO public.users (name, email, password_hash, role)
VALUES ('Duplicate User', 'viraj@gmail.com', 'hash123', 'customer');

ERROR: duplicate key value violates unique constraint "users_email_key"
DETAIL: Key (email)=(viraj@gmail.com) already exists.
```